

TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA CÔNG NGHỆ THÔNG TIN

— o0o —



ĐỒ ÁN LẬP TRÌNH TÍNH TOÁN

XÂY DỰNG CHƯƠNG TRÌNH TÌM ĐƯỜNG ĐI
NGẮN NHẤT TRÊN ĐỒ THỊ CÓ HƯỚNG CÓ
TRỌNG SỐ

Người hướng dẫn: ThS. TRẦN HỒ THỦY TIÊN

Sinh viên thực hiện: Trần Văn Trường Vũ

LỚP: 25Nh.11A

Trần Ngọc Bảo Quyên

LỚP: 25Nh.11A

Đà Nẵng, 06/2026

MỤC LỤC

Mục lục	i
DANH MỤC HÌNH VẼ	ii
MỞ ĐẦU	1
1 TỔNG QUAN ĐỀ TÀI	3
1.1 Bài toán tìm đường đi ngắn nhất và tầm quan trọng	3
1.2 Tổng quan về mô phỏng đồ họa thời gian thực	3
2 CƠ SỞ LÝ THUYẾT	4
2.1 Ý tưởng	4
2.2 Cơ sở lý thuyết	4
2.2.1 Tổng quan về lý thuyết đồ thị (Graph Theory)	4
2.2.2 Lý thuyết đồ thị có hướng (Directed Graph)	4
2.2.3 Lý thuyết đồ họa và tính toán tọa độ vector	5
3 TỔ CHỨC CẤU TRÚC DỮ LIỆU VÀ THUẬT TOÁN	6
3.1 Phát biểu bài toán	6
3.2 Cấu trúc dữ liệu	6
3.3 Thuật toán	6
4 CHƯƠNG TRÌNH VÀ KẾT QUẢ	8
4.1 Tổ chức chương trình	8
4.2 Ngôn ngữ cài đặt	8
4.3 Kết quả	8
4.3.1 Giao diện chính của chương trình	8
4.3.2 Kết quả thực thi của chương trình	9
4.3.3 Nhận xét đánh giá	10
5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	11
5.1 Kết luận	11
5.2 Hướng phát triển	11
TÀI LIỆU THAM KHẢO	12
PHỤ LỤC	13

DANH MỤC HÌNH VẼ

4.1	Giao diện chính giới thiệu thông tin đồ án trên màn hình Console	8
4.2	Giao diện đồ họa phân bố hệ thống các đỉnh theo quỹ đạo đồng tâm	9
4.3	Bảng điều khiển nhật ký hoạt động cập nhật trạng thái loang nhãn từng bước	9
4.4	Biểu diễn lộ trình ngắn nhất bằng hiệu ứng đổi màu cung liên kết sang sắc đỏ	10

MỞ ĐẦU

1. Mục đích thực hiện đề tài

Trong kỷ nguyên công nghệ số và sự phát triển mạnh mẽ của các hệ thống mạng lưới, bài toán tối ưu hóa đường đi trên đồ thị đóng vai trò cốt lõi trong nhiều lĩnh vực thực tiễn. Phần mềm hỗ trợ trực quan hóa giúp công việc thiết kế, phân tích và học tập trở nên tối ưu và thân thiện hơn. Đề tài này được thực hiện nhằm mục đích giúp nhóm sinh viên rèn luyện tư duy thiết kế giải thuật hợp lý, hiện thực hóa các kiến thức cốt lõi đã học trong học phần Toán rời rạc, Kỹ thuật lập trình và Cấu trúc dữ liệu để giải quyết một bài toán thực tiễn phức tạp.

2. Mục tiêu đề tài

Mục tiêu trung tâm của đồ án là phân tích chuyên sâu, thiết kế và thực thi ứng dụng tìm đường đi ngắn nhất giữa hai điểm bất kỳ trên đồ thị có hướng có trọng số. Các mục tiêu cụ thể bao gồm:

- **Xử lý dữ liệu linh hoạt:** Thiết lập module tự động đọc và phân tích cấu trúc ma trận kề từ tệp tin văn bản định dạng tĩnh.
- **Cài đặt thuật toán chính xác:** Áp dụng và tối ưu hóa giải thuật Dijkstra để tính toán nhân đường đi tối ưu, đồng thời lưu vết phục vụ truy xuất lộ trình logic.
- **Mô phỏng đồ họa trực quan:** Tích hợp thư viện đồ họa Raylib để thiết kế không gian tương tác động, biểu diễn quá trình loang duyệt đỉnh một cách sinh động nhất.

3. Phạm vi và đối tượng nghiên cứu

- **Đối tượng nghiên cứu:** Cơ sở toán học của đồ thị có hướng, thuật toán tối ưu hóa nhân Dijkstra, cấu trúc dữ liệu ma trận kề, phương trình hình học tọa độ vector và nguyên lý máy trạng thái (State Machine).
- **Phạm vi nghiên cứu:** Vận dụng tổng hợp ngôn ngữ lập trình C/C++ thuần túy kết hợp thư viện Raylib. Giới hạn trên mô hình đồ thị có trọng số cạnh không âm (điều kiện hội tụ của thuật toán Dijkstra).

4. Phương pháp nghiên cứu

Đồ án vận dụng phương pháp nghiên cứu lý thuyết kết hợp thực nghiệm. Nhóm đã tiến hành khảo sát các giáo trình Toán rời rạc chuyên ngành, nghiên cứu tài liệu lập trình đồ họa tương tác. Từ đó, xây dựng thuật toán, tiến hành code thực nghiệm, đánh giá kết quả chạy trên các tập dữ liệu giả lập và tinh chỉnh logic hiển thị để đạt được trải nghiệm quan sát tốt nhất.

Nhóm sinh viên thực hiện

1. TỔNG QUAN ĐỀ TÀI

1.1. Bài toán tìm đường đi ngắn nhất và tầm quan trọng

Bài toán tìm đường đi ngắn nhất (Shortest Path Problem) là một trong những bài toán tối ưu hóa tổ hợp kinh điển nhất của lý thuyết đồ thị. Bài toán yêu cầu tìm kiếm một lộ trình di chuyển từ điểm bắt đầu u_{start} đến điểm kết thúc v_{target} sao cho tổng chi phí (trọng số) của các cung cấu thành nên lộ trình đó đạt giá trị cực tiểu.

Trong thực tiễn hiện đại, tính ứng dụng của bài toán này là vô cùng rộng lớn. Lời giải của nó là "trái tim" vận hành của hệ thống định vị vệ tinh GPS, hệ thống quản trị bài toán phân phối Logistic chuỗi cung ứng, hệ quy hoạch mạch điện tử bán dẫn, cũng như các giao thức định tuyến luồng gói tin truyền tải trên hạ tầng mạng viễn thông Internet. Việc nắm vững và cài đặt được giải thuật này là bước đệm cơ sở cho mọi kỹ sư phần mềm.

1.2. Tổng quan về mô phỏng đồ họa thời gian thực

Việc học tập và kiểm thử thuật toán trên cửa sổ dòng lệnh (Console) thuần túy thường khô khan và khó phát hiện sai sót logic nội tại. Đồ họa máy tính đóng vai trò thiết yếu trong việc giải quyết vấn đề này. Bằng cách thiết lập một không gian mô phỏng tương tác thời gian thực, toàn bộ cơ chế loang nhãn ẩn sâu bên trong bộ nhớ máy tính sẽ được phơi bày ra màn hình. Người dùng có thể quan sát trực tiếp dòng chảy của thuật toán, sự thay đổi trạng thái của từng phần tử bộ nhớ qua màu sắc đồ họa, từ đó đánh giá chính xác tính hiệu quả của giải thuật.

2. CƠ SỞ LÝ THUYẾT

2.1. Ý tưởng

Ý tưởng cốt lõi của ứng dụng là xây dựng một hệ thống tách biệt luồng xử lý dữ liệu và luồng hiển thị. Chương trình sẽ đọc ma trận đồ thị từ file, sau đó khởi tạo một hệ thống Máy trạng thái (State Machine) để làm chậm quá trình thực thi của thuật toán Dijkstra. Kết hợp với việc quy hoạch tọa độ hình học, mỗi bước tính toán của thuật toán sẽ được đồng bộ hóa với một khung hình đồ họa (frame), tạo ra hiệu ứng chuyển động và đổi màu trực quan.

2.2. Cơ sở lý thuyết

2.2.1. Tổng quan về lý thuyết đồ thị (Graph Theory)

Trước khi tìm hiểu về thuật toán, để hiểu quả điều đầu tiên là nắm vững kiến thức về lý thuyết đồ thị.

Định nghĩa và phân loại

Một đồ thị là một cặp các tập (V, E) , trong đó V là tập các đỉnh và E là tập các cạnh, được biểu diễn bởi cặp đỉnh. tính chất của các mối liên kết (cạnh) giữa các đối tượng (đỉnh) trong bài toán mà đồ thị được phân loại thành:

- Có hướng hoặc vô hướng:
- Có trọng số hoặc không có trọng số:

2.2.2. Lý thuyết đồ thị có hướng (Directed Graph)

Đồ thị có hướng được biểu diễn toán học dưới dạng một hệ vô hướng $G = (V, E)$, trong đó:

- $V = \{v_1, v_2, \dots, v_n\}$ là tập hợp hữu hạn các đỉnh của đồ thị.
- $E \subseteq V \times V$ là tập hợp các cung (cạnh có hướng). Mỗi cung là một cặp có thứ tự (u, v) thể hiện liên kết một chiều từ đỉnh gốc u đến đỉnh ngọn v .
- Hàm trọng số $W : E \rightarrow \mathbb{R}^+$ gán cho mỗi cung (u, v) một giá trị $W(u, v)$ đại diện cho khoảng cách hoặc chi phí.

Đồ thị được mã hóa thành "Ma trận kề" $A_{n \times n}$. Nếu có cung nối từ i đến j , phần tử $A[i][j] = W(i, j)$, ngược lại $A[i][j] = 0$.

2.2.3. Lý thuyết đồ họa và tính toán tọa độ vector

Để hiện thực hóa ứng dụng mô phỏng, hệ thống yêu cầu sự can thiệp của hình học tính toán. Để các đỉnh không đè lên nhau, đồ án áp dụng phương trình tham số đường tròn, dàn đều n đỉnh quanh một tâm $I(x_0, y_0)$ với bán kính R :

$$x_i = x_0 + R \cdot \cos \left((i - 1) \frac{2\pi}{n} \right) \quad (2.1)$$

$$y_i = y_0 + R \cdot \sin \left((i - 1) \frac{2\pi}{n} \right) \quad (2.2)$$

Để vẽ mũi tên hướng mà phần nhọn không bị lọt vào tâm đỉnh đích V , chương trình sử dụng hàm `atan2f` để xác định góc dốc θ . Điểm dừng của mũi tên được tịnh tiến lùi lại một đoạn chính bằng bán kính nút tròn.

3. TỔ CHỨC CẤU TRÚC DỮ LIỆU VÀ THUẬT TOÁN

3.1. Phát biểu bài toán

- **Dữ liệu đầu vào:** Hệ thống nhận tệp cấu trúc văn bản lưu trữ ma trận kề A kích thước $n \times n$. Kèm theo là chỉ số của hai nút bắt đầu và kết thúc lộ trình truy vấn.
- **Yêu cầu đầu ra:** Cửa sổ đồ họa 60-FPS hiển thị quá trình đổi màu động của các đỉnh (Vàng: Đang xét, Xanh: Đã tối ưu) và bảng điều khiển Log ghi nhận từng dòng trạng thái toán học.

3.2. Cấu trúc dữ liệu

Hệ thống đóng gói toàn bộ trạng thái lý thuyết vào một `struct Graph` tĩnh, sử dụng hệ thống mảng một chiều tối ưu bộ nhớ cache để xử lý tốc độ cao:

```

1 #define N 105
2 #define INF 1000000000
3
4 struct Graph {
5     int n;                // Số lượng đỉnh
6     int A[N][N];          // Ma trận kề trong sơ
7     Vector2 toado[N];     // Tọa độ trục quan của đỉnh trên UI
8     int L[N];             // Mảng khoảng cách ngắn nhất (Nhân tam thời)
9     int T[N];             // Trạng thái tập hợp T (1: Chưa có đỉnh, 0: Đã
10    co đỉnh)
11    int par[N];            // Mảng lưu vết cha truy xuan lo trình
12 };

```

Listing 3.1: Định nghĩa cấu trúc đồ thị tĩnh

3.3. Thuật toán

Thuật toán Dijkstra được sử dụng làm lõi xử lý. Tuy nhiên, để tương thích với giao diện đồ họa, thuật toán được thiết kế lại dưới dạng Máy trạng thái (State Machine) điều khiển bởi biến `AlgoState`:

- **STATE_FIND_MIN:** Trạng thái quét tìm đỉnh trong đồ thị có $L[i]$ nhỏ nhất. Đỉnh này được chọn làm tâm loang.

- **STATE_RELAX_EDGES**: Chuyển sang trạng thái cập nhật nhãn (Relaxation). Xét mọi đỉnh v kề liền sau u . Nếu $L[u] + W(u, v) < L[v]$, tiến hành tối ưu hóa khoảng cách và lưu vết đỉnh cha.
- **STATE_FINISHED**: Kích hoạt khi tìm thấy đích. Truy ngược mảng **par** để xuất lộ trình cuối cùng.

4. CHƯƠNG TRÌNH VÀ KẾT QUẢ

4.1. Tổ chức chương trình

Chương trình được phân rã chức năng mạch lạc thành các phân vùng module tuyến tính:

- Module quản lý File (`inputdata`): Phân tích tệp `input.dat`.
- Module Hình học đồ họa (`drawArrow`): Xử lý vector vẽ mũi tên hướng và gán trọng số.
- Module Động cơ trạng thái (`UIRaylib_Dijkstra`): Quản lý bộ đếm thời gian và cập nhật vòng lặp 60 khung hình/giây.

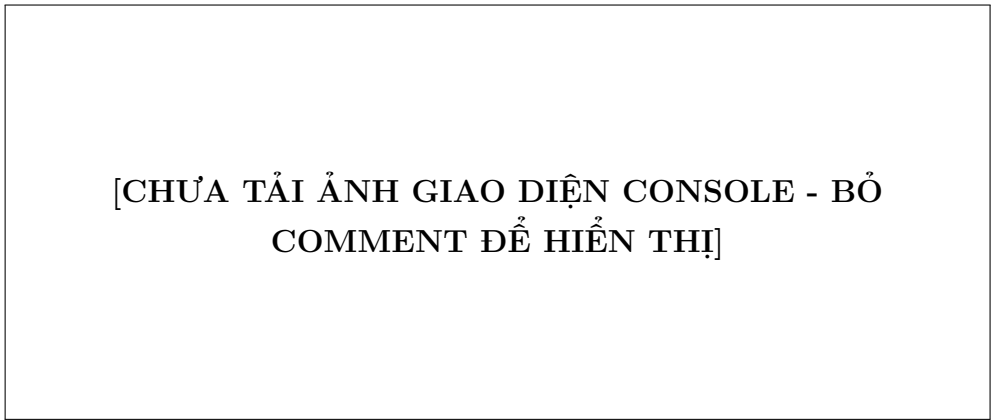
4.2. Ngôn ngữ cài đặt

Hệ thống được lập trình bằng ngôn ngữ C/C++ trên nền tảng trình biên dịch GCC (MinGW). Công cụ xây dựng chương trình được cấu hình tự động thông qua ‘Makefile’. Giao diện tương tác được hiện thực hóa bằng bộ thư viện mã nguồn mở Raylib.

4.3. Kết quả

4.3.1. Giao diện chính của chương trình

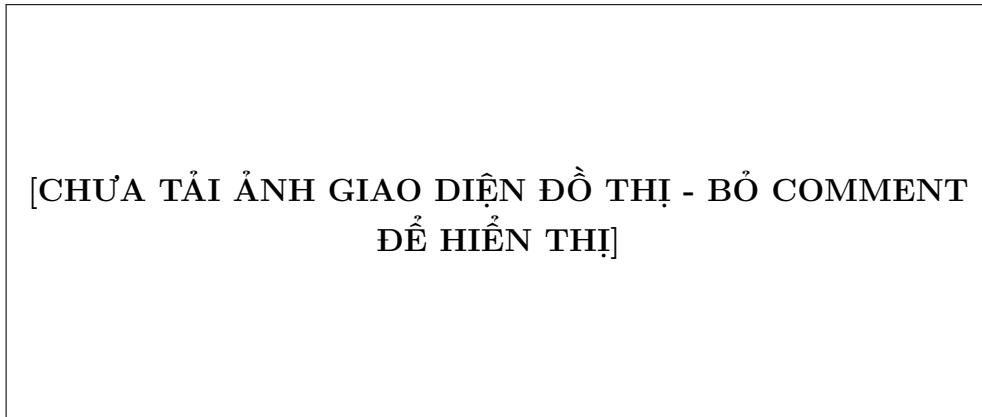
Màn hình khởi tạo (Console) đảm nhận vai trò hiển thị thông tin sinh viên thực hiện, giới thiệu hệ thống và xác nhận việc nạp tệp đồ thị thành công.



[CHƯA TẢI ẢNH GIAO DIỆN CONSOLE - BỎ
COMMENT ĐỂ HIỂN THỊ]

Hình 4.1: Giao diện chính giới thiệu thông tin đồ án trên màn hình Console

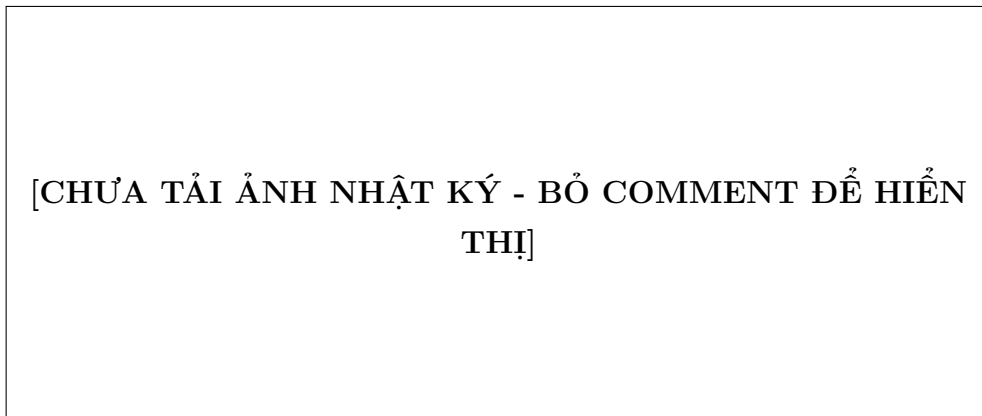
Sau khi nạp tệp thành công, hệ thống đồ họa Raylib sẽ khởi chạy với sân khấu phân bố các nút đỉnh đều theo cấu trúc elip tự động, đảm bảo thẩm mỹ thị giác.



Hình 4.2: Giao diện đồ họa phân bố hệ thống các đỉnh theo quỹ đạo đồng tâm

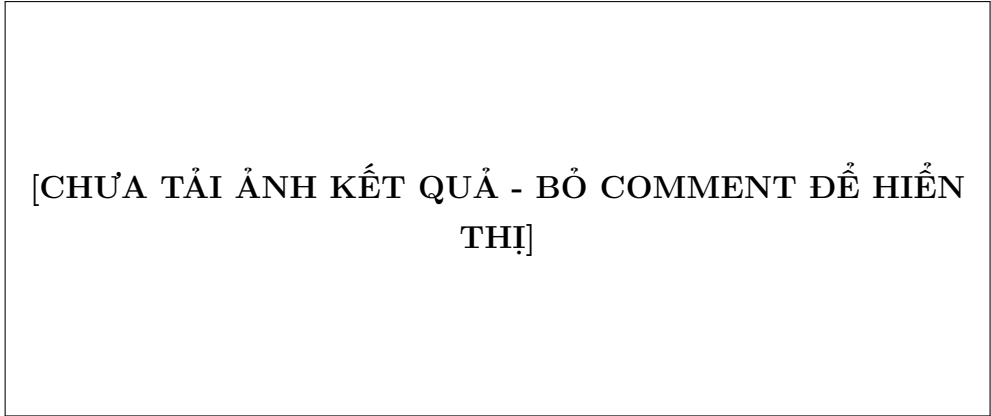
4.3.2. Kết quả thực thi của chương trình

Trong suốt quá trình phần mềm thực thi, nhãn thông tin khoảng cách L cập nhật liên tục dưới chân từng nút tròn. Sắc xanh lá đại diện cho đỉnh đã tối ưu hoàn toàn, sắc vàng biểu thị đỉnh đang đóng vai trò tâm loang xét duyệt hiện tại. Bất kỳ sự thay đổi nào cũng được ghi nhận vào bảng Log bên phải.



Hình 4.3: Bảng điều khiển nhật ký hoạt động cập nhật trạng thái loang nhãn từng bước

Khi tìm thấy đích, toàn bộ các cạnh và đỉnh cấu thành đường đi ngắn nhất tự động chuyển đổi sắc màu hiển thị sang màu đỏ rực rỡ với độ dày nét vẽ vượt trội nhằm cô lập lộ trình trực quan cho người sử dụng.



[CHƯA TẢI ẢNH KẾT QUẢ - BỎ COMMENT ĐỂ HIỂN THỊ]

Hình 4.4: Biểu diễn lộ trình ngắn nhất bằng hiệu ứng đổi màu cung liên kết sang sắc đỏ

4.3.3. Nhận xét đánh giá

Ưu điểm: Ứng dụng giải quyết triệt để bài toán tìm kiếm tối ưu trên mô hình đồ thị có hướng phức tạp. Thiết kế giao diện đồ họa có tính tương tác cao, cơ chế máy trạng thái phân rã logic vận hành trơn tru và trực quan hóa sâu sắc dữ liệu từng bước chạy. Không xảy ra rò rỉ bộ nhớ (Memory Leak) nhờ sử dụng hạ tầng mảng cấp phát tĩnh.

Hạn chế tồn tại:

- Chương trình hiện tại yêu cầu dữ liệu tệp tin văn bản đầu vào phải tuân thủ nghiêm ngặt cấu trúc định dạng chuẩn hóa sẵn, chưa có cơ chế kiểm lỗi ngoại lệ mạnh mẽ.
- Thao tác tìm kiếm đỉnh Min hiện tại đang dùng cấu trúc duyệt tuyến tính $O(V)$ cho mỗi bước. Do đó tổng độ phức tạp thuật toán đang ở ngưỡng $O(V^2)$. Đây là điểm yếu nếu đồ thị có hàng chục ngàn đỉnh.

5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Qua quá trình nỗ lực nghiên cứu và hoàn thiện đồ án PBL1, nhóm sinh viên đã củng cố thành công nền tảng cấu trúc dữ liệu tĩnh và làm chủ giải thuật cốt lõi Dijkstra. Việc áp dụng thành công kỹ thuật hình học lượng giác và thư viện Raylib vào kiến trúc Máy trạng thái phân mảnh đã giúp chương trình vận hành chính xác, minh họa trực quan sinh động tiến trình loang nhãn phức tạp của hệ thống đồ thị.

5.2. Hướng phát triển

- Khắc phục độ phức tạp $O(V^2)$ bằng cách nâng cấp cấu trúc mảng duyệt tuần tự thành Hàng đợi ưu tiên (Min-Heap), giúp giảm độ phức tạp thời gian về ngưỡng $O(E \log V)$.
- Phát triển Module kéo thả tương tác chuột trên màn hình, cho phép người dùng tự do vẽ đỉnh, nối cung đồ thị bằng thao tác Click thay vì phải nhập liệu tệp văn bản.
- Tối ưu hoá hình vẽ đồ thị cho dữ liệu lớn, minh họa rõ ràng, trực quan hơn.
- Tích hợp song song giải thuật Bellman-Ford để so sánh trực quan thời gian chạy và khả năng xử lý đồ thị chứa trọng số âm.

TÀI LIỆU THAM KHẢO

- [1] Nguyễn Đức Nghĩa, *Giáo trình Toán rời rạc*, Nhà xuất bản Bách Khoa Hà Nội.
- [2] Phan Thanh Tao, *Giáo trình Toán rời rạc*, Trường Đại học Bách Khoa – Đại học Đà Nẵng.
- [3] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, MIT Press.
- [4] Raylib Graph Graphics Development Documentation, <https://www.raylib.com/>.
- [5] Nguyễn Văn Hiệu, *Bài toán đường đi ngắn nhất trên đồ thị*, Trường Đại học Bách Khoa - Đại học Đà Nẵng.
- [6] Nguyễn Văn Hiệu, *Tập bài giảng Toán rời rạc*, Trường Đại học Bách Khoa - Đại học Đà Nẵng.

PHỤ LỤC

MÃ NGUỒN DỰ ÁN (GITHUB)

Toàn bộ mã nguồn, cách cài đặt, cách sử dụng đồ án được lưu trữ tại:

<https://github.com/raii-cod/PBL-1>

Trích đoạn các hàm xử lý cốt lõi của ứng dụng

1. Động cơ phân mảnh Máy trạng thái Dijkstra:

```
1 if (state == STATE_FIND_MIN) {
2     int min_d = INF; currentV = -1;
3     for (int i = 1; i <= g.n; i++)
4         if (g.T[i] && g.L[i] < min_d) { min_d = g.L[i]; currentV = i; }
5
6     if (currentV == -1 || currentV == v_target) {
7         state = STATE_FINISHED;
8         if (g.L[v_target] != INF) {
9             int t = v_target;
10            int temp[N], count = 0;
11            while (t != -1) { onPath[t] = true; temp[count++] = t; t = g
.par[t]; }
12        }
13        AddLog(">>> KET THUC: Da xac dinh lo trinh.", stepLogs, &
logCount);
14    } else {
15        g.T[currentV] = 0; neighborIdx = 1; state = STATE_RELAX_EDGES;
16    }
17 }
18 else if (state == STATE_RELAX_EDGES) {
19     bool found = false;
20     while (neighborIdx <= g.n) {
21         if (g.A[currentV][neighborIdx] != -1 && g.T[neighborIdx]) {
22             int newD = g.L[currentV] + g.A[currentV][neighborIdx];
23             if (newD < g.L[neighborIdx]) {
24                 g.L[neighborIdx] = newD;
25                 g.par[neighborIdx] = currentV;
26             }
27             neighborIdx++; found = true; break;

```



```
28     }
29     neighborIdx++;
30 }
31 if (!found) state = STATE_FIND_MIN;
32 }
```

2. Hàm tính toán vector dựng mũi tên:

```
1 void drawArrow(Vector2 start, Vector2 end, float rDinh, Color mau, int
   trongSo) {
2     float angle = atan2f(end.y - start.y, end.x - start.x);
3     Vector2 tip = { end.x - (rDinh + 5) * cosf(angle), end.y - (rDinh +
   5) * sinf(angle) };
4     DrawLineEx(start, tip, 2.0f, mau);
5
6     float arrowSize = 14.0f;
7     Vector2 p1 = { tip.x + arrowSize * cosf(angle + PI - 0.45f), tip.y +
   arrowSize * sinf(angle + PI - 0.45f) };
8     Vector2 p2 = { tip.x + arrowSize * cosf(angle + PI + 0.45f), tip.y +
   arrowSize * sinf(angle + PI + 0.45f) };
9     DrawTriangle(tip, p2, p1, mau);
10 }
```